

This document describes the Application Programming Interface (API) for the data structures and algorithms discussed in the book *Algorithms*  by Robert Sedgewick and Kevin Wayne. The corresponding code is part of a package called `dsa`.

Contents

1	Object-oriented Programming	3
1.1	Counter implements Comparable<Counter>	3
1.2	Date implements Comparable<Date>	3
1.3	Point2D implements Comparable<Point2D>	3
1.4	RectHV	4
1.5	Transaction implements Comparable<Transaction>	4
2	Algorithms and Data Structures	5
2.1	LinearSearch	5
2.2	BinarySearch	5
2.3	interface Bag<T> extends Iterable<T>	5
2.4	LinkedBag<T> implements Bag<T>	5
2.5	ResizingArrayBag<T> implements Bag<T>	6
2.6	interface Queue<T> extends Iterable<T>	6
2.7	LinkedQueue<T> implements Queue<T>	6
2.8	ResizingArrayQueue<T> implements Queue<T>	6
2.9	interface Stack<T> extends Iterable<T>	7
2.10	LinkedStack<T> implements Stack<T>	7
2.11	ResizingArrayStack<T> implements Stack<T>	7
2.12	interface UF	8
2.13	QuickFindUF implements UF	9
2.14	QuickUnionUF implements UF	9
2.15	WeightedQuickUnionUF implements UF	9
2.16	WeightedQuickUnionPathCompressionUF implements UF	9
3	Sorting	9
3.1	Bubble, Selection, Insertion, Shell, Merge, Quick, Quick3way, Heap	9
3.2	MinPQ<T> implements Iterable<T>	10
3.3	MaxPQ<T> implements Iterable<T>	10
3.4	IndexMinPQ<T> extends Comparable<T> implements Iterable<T>	10
3.5	IndexMaxPQ<T> extends Comparable<T> implements Iterable<T>	11
3.6	Inversions	11
4	Searching	11
4.1	interface BasicST<K, V>	11
4.2	interface OrderedST<K extends Comparable<K>, V>	12
4.3	LinearSearchST<K, V> implements BasicST<K, V>	12
4.4	BinarySearchST<K extends Comparable<K>, V> implements OrderedST<K, V>	12
4.5	SeparateChainingHashST<K, V> implements BasicST<K, V>	13
4.6	BinarySearchTreeST<K extends Comparable<K>, V> implements OrderedST<K, V>	13
4.7	RedBlackBinarySearchTreeST<K extends Comparable<K>, V> implements OrderedST<K, V>	14
4.8	Set<K extends Comparable<K>> implements Iterable<K>	14
4.9	SparseVector	15
4.10	SparseMatrix	15

5	Graphs	15
5.1	Graph	15
5.2	interface Paths	16
5.3	DFSPaths implements Paths	16
5.4	BFSPaths implements Paths	16
5.5	SymbolGraph	16
5.6	DiGraph	16
5.7	DFSPaths implements Paths	17
5.8	BFSDiPaths implements Paths	17
5.9	SymbolDiGraph	17
5.10	DiCycle	17
5.11	DFSOrders	17
5.12	Topological	18
5.13	Edge implements Comparable<Edge>	18
5.14	EdgeWeightedGraph	18
5.15	Kruskal	18
5.16	DiEdge	18
5.17	EdgeWeightedDiGraph	19
5.18	Dijkstra implements Paths	19
6	Strings	19
6.1	Alphabet	19
6.2	LSD	20
6.3	MSD	20
6.4	TrieST	20
6.5	KMP	20
6.6	NFA	20
6.7	Genome	21
6.8	RunLength	21
6.9	Huffman	21

1 Object-oriented Programming

1.1 Counter implements Comparable<Counter>

Counter(String id)	constructs a counter given its id
void increment()	increments this counter by 1
int tally()	returns the current value of this counter
void reset()	resets this counter to zero
boolean equals(Object other)	returns true if this counter and other have the same tally, and false otherwise
String toString()	returns a string representation of this counter
int compareTo(Counter other)	returns a comparison of this counter with other by their tally

1.2 Date implements Comparable<Date>

Date(int month, int day, int year)	constructs a date from month , day , and year
Date(String s)	constructs a date from a string s of the form “MM/DD/YYYY”
int month()	returns the month (an integer between 1 and 12)
int day()	returns the day (an integer between 1 and 31)
int year()	returns the year
Date next()	returns the next date in the calendar
boolean isBefore(Date other)	returns true if this date is before other , and false otherwise
boolean isAfter(Date other)	returns true if this date is after other , and false otherwise
boolean equals(Object other)	returns true if this date is the same as other , and false otherwise
int hashCode()	returns a hash code for this date
String toString()	returns a string representation of this date
int compareTo(Date other)	returns a chronological comparison of this date with other

1.3 Point2D implements Comparable<Point2D>

Point2D(double x, double y)	constructs a point with the given (x , y)
double x()	returns the <i>x</i> -coordinate of this point
double y()	returns the <i>y</i> -coordinate of this point
double r()	returns the polar radius of this point
double theta()	returns the polar angle ($-\pi$ to π) of this point
double distanceTo(Point2D other)	returns the Euclidean distance between this point and other
double distanceSquaredTo(Point2D other)	returns the squared Euclidean distance between this point and other
boolean equals(Object other)	returns true if this point and other have the same coordinates, and false otherwise
int hashCode()	returns the hash code for this point
String toString()	returns a string representation of this point
void draw()	draws this point using standard draw
void draw(Point2D other)	draws a line between this point and other using standard draw
int compareTo(Point2D other)	returns a comparison of this point with other , by their <i>x</i> - and <i>y</i> -coordinates

<code>Comparator<Point2D> atan2Order()</code>	returns a comparator for comparing two points by the arctan angle ($-\pi$ to π) they make with this point
<code>Comparator<Point2D> polarOrder()</code>	returns a comparator for comparing two points by the polar angle (0 to 2π) they make with this point
<code>Comparator<Point2D> distanceToOrder()</code>	returns a comparator for comparing two points by their distance to this point
<code>static Comparator<Point2D> xOrder()</code>	returns a comparator for comparing two points by their x -coordinate
<code>static Comparator<Point2D> yOrder()</code>	returns a comparator for comparing two points by their y -coordinate
<code>static Comparator<Point2D> rOrder()</code>	returns a comparator for comparing two points by their polar radius

1.4 RectHV

<code>RectHV(double xMin, double yMin, double xMax, double yMax)</code>	constructs a rectangle $[xMin, yMin]$ x $[xMax, yMax]$
<code>double xMin()</code>	returns the minimum x -coordinate of any point in this rectangle
<code>double yMin()</code>	returns the minimum y -coordinate of any point in this rectangle
<code>double xMax()</code>	returns the maximum x -coordinate of any point in this rectangle
<code>double yMax()</code>	returns the maximum y -coordinate of any point in this rectangle
<code>double width()</code>	returns the width of this rectangle
<code>double height()</code>	returns the height of this rectangle
<code>boolean intersects(RectHV other)</code>	returns true if this rectangle intersects with other , and false otherwise
<code>boolean contains(Point2D p)</code>	returns true if this rectangle contains the point p , and false otherwise
<code>double distanceTo(Point2D p)</code>	returns the Euclidean distance between the point p and the closest point on this rectangle, and 0 if the point is within
<code>double distanceSquaredTo(Point2D p)</code>	returns the squared Euclidean distance between the point p and the closest point on this rectangle, and 0 if the point is within
<code>boolean equals(Object other)</code>	returns true if this rectangle and other have the same x - and y -bounds, and false otherwise
<code>int hashCode()</code>	returns the hash code for this rectangle
<code>String toString()</code>	returns a string representation of this rectangle
<code>void draw()</code>	draws this rectangle using standard draw

1.5 Transaction implements Comparable<Transaction>

<code>Transaction(String name, Date date, double amount)</code>	constructs a transaction from a name , date , and amount
<code>Transaction(String s)</code>	constructs a transaction from a string s of the form “name date amount”
<code>String name()</code>	returns the name of the person involved in this transaction

<code>Date date()</code>	returns the date of this transaction
<code>double amount()</code>	returns the amount of this transaction
<code>int hashCode()</code>	returns a hash code for this transaction
<code>String toString()</code>	returns a string representation of this transaction
<code>int compareTo(Transaction other)</code>	returns a comparison of this transaction with other by amount
<code>static Comparator<Transaction> nameOrder()</code>	returns a comparator for comparing two transactions by name
<code>static Comparator<Transaction> dateOrder()</code>	returns a comparator for comparing two transactions by date

2 Algorithms and Data Structures

2.1 LinearSearch

<code>static int indexOf(Object[] a, Object item)</code>	returns the index of item in the array a , or -1
<code>static int indexOf(int[] a, int item)</code>	returns the index of item in the array a , or -1
<code>static int indexOf(double[] a, double item)</code>	returns the index of item in the array a , or -1

2.2 BinarySearch

<code>static <T extends Comparable<T>> int indexOf(T[] a, T item)</code>	returns the index of item in the sorted array a , or -1
<code>static int indexOf(T[] a, T item, Comparator<T> c)</code>	returns the index of item in the sorted array a , or -1 (comparisons are made using c)
<code>static int indexOf(int[] a, int item)</code>	returns the index of item in the sorted array a , or -1
<code>static int indexOf(double[] a, double item)</code>	returns the index of item in the sorted array a , or -1

2.3 interface Bag<T> extends Iterable<T>

<code>boolean isEmpty()</code>	returns true if this bag is empty, and false otherwise
<code>int size()</code>	returns the number of items in this bag
<code>void add(T item)</code>	adds item to this bag
<code>String toString()</code>	returns a string representation of this bag
<code>Iterator<T> iterator()</code>	returns an iterator to iterate over the items in this bag

2.4 LinkedBag<T> implements Bag<T>

<code>LinkedBag()</code>	constructs an empty bag
<code>boolean isEmpty()</code>	returns true if this bag is empty, and false otherwise
<code>int size()</code>	returns the number of items in this bag
<code>void add(T item)</code>	adds <code>item</code> to this bag
<code>String toString()</code>	returns a string representation of this bag
<code>Iterator<T> iterator()</code>	returns an iterator to iterate over the items in this bag

2.5 ResizingArrayBag<T> implements Bag<T>

<code>ResizingArrayBag()</code>	constructs an empty bag
<code>boolean isEmpty()</code>	returns true if this bag is empty, and false otherwise
<code>int size()</code>	returns the number of items in this bag
<code>void add(T item)</code>	adds <code>item</code> to this bag
<code>String toString()</code>	returns a string representation of this bag
<code>Iterator<T> iterator()</code>	returns an iterator to iterate over the items in this bag

2.6 interface Queue<T> extends Iterable<T>

<code>boolean isEmpty()</code>	returns true if this queue is empty, and false otherwise
<code>int size()</code>	returns the number of items in this queue
<code>void enqueue(Item item)</code>	adds <code>item</code> to the end of this queue
<code>Item peek()</code>	returns the item at the front of this queue
<code>Item dequeue()</code>	removes and returns the item at the front of this queue
<code>String toString()</code>	returns a string representation of this queue
<code>Iterator<T> iterator()</code>	returns an iterator to iterate over the items in this queue in FIFO order

2.7 LinkedQueue<T> implements Queue<T>

<code>LinkedQueue()</code>	constructs an empty queue
<code>boolean isEmpty()</code>	returns true if this queue is empty, and false otherwise
<code>int size()</code>	returns the number of items in this queue
<code>void enqueue(Item item)</code>	adds <code>item</code> to the end of this queue
<code>Item peek()</code>	returns the item at the front of this queue
<code>Item dequeue()</code>	removes and returns the item at the front of this queue
<code>String toString()</code>	returns a string representation of this queue
<code>Iterator<T> iterator()</code>	returns an iterator to iterate over the items in this queue in FIFO order

2.8 ResizingArrayQueue<T> implements Queue<T>

<code>ResizingArrayQueue()</code>	constructs an empty queue
<code>boolean isEmpty()</code>	returns true if this queue is empty, and false otherwise
<code>int size()</code>	returns the number of items in this queue
<code>void enqueue(Item item)</code>	adds <code>item</code> to the end of this queue
<code>Item peek()</code>	returns the item at the front of this queue
<code>Item dequeue()</code>	removes and returns the item at the front of this queue
<code>String toString()</code>	returns a string representation of this queue
<code>Iterator<T> iterator()</code>	returns an iterator to iterate over the items in this queue in FIFO order

2.9 interface Stack<T> extends Iterable<T>

<code>boolean isEmpty()</code>	returns true if this stack is empty, and false otherwise
<code>int size()</code>	returns the number of items in this stack
<code>void push(Item item)</code>	adds <code>item</code> to the top of this stack
<code>Item peek()</code>	returns the item at the top of this stack
<code>Item pop()</code>	removes and returns the item at the top of this stack
<code>String toString()</code>	returns a string representation of this stack
<code>Iterator<T> iterator()</code>	returns an iterator to iterate over the items in this stack in LIFO order

2.10 LinkedStack<T> implements Stack<T>

<code>LinkedStack()</code>	constructs an empty stack
<code>boolean isEmpty()</code>	returns true if this stack is empty, and false otherwise
<code>int size()</code>	returns the number of items in this stack
<code>void push(Item item)</code>	adds <code>item</code> to the top of this stack
<code>Item peek()</code>	returns the item at the top of this stack
<code>Item pop()</code>	removes and returns the item at the top of this stack
<code>String toString()</code>	returns a string representation of this stack
<code>Iterator<T> iterator()</code>	returns an iterator to iterate over the items in this stack in LIFO order

2.11 ResizingArrayStack<T> implements Stack<T>

<code>ResizingArrayStack()</code>	constructs an empty stack
<code>boolean isEmpty()</code>	returns true if this stack is empty, and false otherwise
<code>int size()</code>	returns the number of items in this stack
<code>void push(Item item)</code>	adds <code>item</code> to the top of this stack
<code>Item peek()</code>	returns the item at the top of this stack
<code>Item pop()</code>	removes and returns the item at the top of this stack
<code>String toString()</code>	returns a string representation of this stack
<code>Iterator<T> iterator()</code>	returns an iterator to iterate over the items in this stack in LIFO order

2.12 interface UF

<code>int find(int p)</code>	returns the canonical site of the component containing site <code>p</code>
<code>int count()</code>	returns the number of components
<code>boolean connected(int p, int q)</code>	returns <code>true</code> if sites <code>p</code> and <code>q</code> belong to the same component, and <code>false</code> otherwise
<code>void union(int p, int q)</code>	connects sites <code>p</code> and <code>q</code>

2.13 QuickFindUF implements UF

<code>QuickFindUF(int n)</code>	constructs an empty union-find data structure with n sites
<code>int find(int p)</code>	returns the canonical site of the component containing site p
<code>int count()</code>	returns the number of components
<code>boolean connected(int p, int q)</code>	returns true if sites p and q belong to the same component, and false otherwise
<code>void union(int p, int q)</code>	connects sites p and q

2.14 QuickUnionUF implements UF

<code>QuickUnionUF(int n)</code>	constructs an empty union-find data structure with n sites
<code>int find(int p)</code>	returns the canonical site of the component containing site p
<code>int count()</code>	returns the number of components
<code>boolean connected(int p, int q)</code>	returns true if sites p and q belong to the same component, and false otherwise
<code>void union(int p, int q)</code>	connects sites p and q

2.15 WeightedQuickUnionUF implements UF

<code>WeightedQuickUnionUF(int n)</code>	constructs an empty union-find data structure with n sites
<code>int find(int p)</code>	returns the canonical site of the component containing site p
<code>int count()</code>	returns the number of components
<code>boolean connected(int p, int q)</code>	returns true if sites p and q belong to the same component, and false otherwise
<code>void union(int p, int q)</code>	connects sites p and q

2.16 WeightedQuickUnionPathCompressionUF implements UF

<code>WeightedQuickUnionPathCompressionUF(int n)</code>	constructs an empty union-find data structure with n sites
<code>int find(int p)</code>	returns the canonical site of the component containing site p
<code>int count()</code>	returns the number of components
<code>boolean connected(int p, int q)</code>	returns true if sites p and q belong to the same component, and false otherwise
<code>void union(int p, int q)</code>	connects sites p and q

3 Sorting

3.1 Bubble, Selection, Insertion, Shell, Merge, Quick, Quick3way, Heap

<code>static <T extends Comparable<T>> void sort(T[] a)</code>	sorts the array a according to the natural order of its objects
<code>static <T> void sort(T[] a, Comparator<T> c)</code>	sorts the array a according to the order induced by the comparator c
<code>static void sort(int[] a)</code>	sorts the array a
<code>static void sort(double[] a)</code>	sorts the array a

3.2 MinPQ<T> implements Iterable<T>

MinPQ()	constructs an empty minPQ
MinPQ(Comparator<T> c)	constructs an empty minPQ with the given comparator
MinPQ(int capacity)	constructs an empty minPQ with the given capacity
MinPQ(int capacity, Comparator<T> c)	constructs an empty minPQ with the given capacity and comparator
boolean isEmpty()	returns true if this minPQ is empty, and false otherwise
int size()	returns the number of items in this minPQ
void insert(T item)	adds item to this minPQ
T min()	returns the smallest item in this minPQ
T deleteMin()	removes and returns the smallest item in this minPQ
String toString()	returns a string representation of this minPQ
Iterator<T> iterator()	returns an iterator to iterate over the items in this minPQ in ascending order

3.3 MaxPQ<T> implements Iterable<T>

MaxPQ()	constructs an empty maxPQ
MaxPQ(Comparator<T> c)	constructs an empty maxPQ with the given comparator
MaxPQ(int capacity)	constructs an empty maxPQ with the given capacity
MaxPQ(int capacity, Comparator<T> c)	constructs an empty maxPQ with the given capacity and comparator
boolean isEmpty()	returns true if this maxPQ is empty, and false otherwise
int size()	returns the number of items in this maxPQ
void insert(T item)	adds item to this maxPQ
T max()	returns the largest item in this maxPQ
T deleteMax()	removes and returns the largest item in this maxPQ
String toString()	returns a string representation of this maxPQ
Iterator<T> iterator()	returns an iterator to iterate over the items in this maxPQ in descending order

3.4 IndexMinPQ<T> extends Comparable<T>> implements Iterable<T>

IndexMinPQ(int maxN)	constructs an empty indexMinPQ with indices from the interval [0, maxN)
boolean isEmpty()	returns true if this indexMinPQ is empty, and false otherwise
int size()	returns the number of items in this indexMinPQ
void insert(int i, T item)	associates item with index i in this indexMinPQ
void change(int i, T item)	changes the item associated with index i to item in this indexMinPQ
boolean contains(int i)	returns true if i is an index in this indexMinPQ, and false otherwise
int minIndex()	returns the index associated with the smallest item in this indexMinPQ
T minItem()	returns the smallest item in this indexMinPQ
T itemAt(int i)	returns the item associated with index i in this indexMinPQ
int deleteMin()	removes the smallest item from this indexMinPQ and returns its associated index
void delete(int i)	removes the item associated with index i in this indexMinPQ

<code>String toString()</code>	returns a string representation of this <code>indexMinPQ</code>
<code>Iterator<Integer> iterator()</code>	returns an iterator to iterate over the indices in this <code>indexMinPQ</code> in ascending order of the associated items

3.5 `IndexMaxPQ<T>` extends `Comparable<T>>` implements `Iterable<T>`

<code>IndexMaxPQ(int maxN)</code>	constructs an empty <code>indexMaxPQ</code> with indices from the interval <code>[0, maxN)</code>
<code>boolean isEmpty()</code>	returns <code>true</code> if this <code>indexMaxPQ</code> is empty, and <code>false</code> otherwise
<code>int size()</code>	returns the number of items in this <code>indexMaxPQ</code>
<code>void insert(int i, T item)</code>	associates <code>item</code> with index <code>i</code> in this <code>indexMaxPQ</code>
<code>void change(int i, T item)</code>	changes the item associated with index <code>i</code> to <code>item</code> in this <code>indexMaxPQ</code>
<code>boolean contains(int i)</code>	returns <code>true</code> if <code>i</code> is an index in this <code>indexMaxPQ</code> , and <code>false</code> otherwise
<code>int maxIndex()</code>	returns the index associated with the largest item in this <code>indexMaxPQ</code>
<code>T maxItem()</code>	returns the largest item in this <code>indexMaxPQ</code>
<code>T itemAt(int i)</code>	returns the item associated with index <code>i</code> in this <code>indexMaxPQ</code>
<code>int deleteMax()</code>	removes the largest item from this <code>indexMaxPQ</code> and returns its associated index
<code>void delete(int i)</code>	removes the item associated with index <code>i</code> in this <code>indexMaxPQ</code>
<code>String toString()</code>	returns a string representation of this <code>indexMaxPQ</code>
<code>Iterator<Integer> iterator()</code>	returns an iterator to iterate over the indices in this <code>indexMaxPQ</code> in descending order of the associated items

3.6 Inversions

<code>static long count(Comparable[] a)</code>	returns the number of inversions in the array <code>a</code> according to the natural order of its objects
<code>static long count(Object[] a, Comparator c)</code>	returns the number of inversions in the array <code>a</code> according to the order induced by the comparator <code>c</code>
<code>static long count(int a[])</code>	returns the number of inversions in the array <code>a</code>
<code>static long count(double a[])</code>	returns the number of inversions in the array <code>a</code>

4 Searching

4.1 `interface BasicST<K, V>`

<code>boolean isEmpty()</code>	returns <code>true</code> if this symbol table is empty, and <code>false</code> otherwise
<code>int size()</code>	returns the number of key-value pairs in this symbol table
<code>void put(K key, V value)</code>	inserts the <code>key</code> and <code>value</code> pair into this symbol table
<code>V get(K key)</code>	returns the value associated with <code>key</code> in this symbol table, or <code>null</code>
<code>boolean contains(K key)</code>	returns <code>true</code> if this symbol table contains <code>key</code> , and <code>false</code> otherwise
<code>void delete(K key)</code>	deletes <code>key</code> and the associated value from this symbol table
<code>Iterable<K> keys()</code>	returns all the keys in this symbol table
<code>String toString()</code>	returns a string representation of this symbol table

4.2 interface OrderedST<K extends Comparable<K>, V>

<code>boolean isEmpty()</code>	returns true if this symbol table is empty, and false otherwise
<code>int size()</code>	returns the number of key-value pairs in this symbol table
<code>void put(K key, V value)</code>	inserts the key and value pair into this symbol table
<code>V get(K key)</code>	returns the value associated with key in this symbol table, or null
<code>boolean contains(K key)</code>	returns true if this symbol table contains key , and false otherwise
<code>void delete(K key)</code>	deletes key and the associated value from this symbol table
<code>Iterable<K> keys()</code>	returns all the keys in this symbol table in sorted order
<code>K min()</code>	returns the smallest key in this symbol table
<code>K max()</code>	returns the largest key in this symbol table
<code>void deleteMin()</code>	deletes the smallest key and the associated value from this symbol table
<code>void deleteMax()</code>	deletes the largest key and the associated value from this symbol table
<code>K floor(K key)</code>	returns the largest key in this symbol table that is smaller than or equal to key
<code>K ceiling(K key)</code>	returns the smallest key in this symbol table that is greater than or equal to key
<code>int rank(K key)</code>	returns the number of keys in this symbol table that are strictly smaller than key
<code>K select(int k)</code>	returns the key in this symbol table with the rank k
<code>int size(K lo, K hi)</code>	returns the number of keys in this symbol table that are in the interval [lo, hi]
<code>Iterable<K> keys(K lo, K hi)</code>	returns the keys in this symbol table that are in the interval [lo, hi] in sorted order
<code>String toString()</code>	returns a string representation of this symbol table

4.3 LinearSearchST<K, V> implements BasicST<K, V>

<code>LinearSearchST()</code>	constructs an empty symbol table
<code>boolean isEmpty()</code>	returns true if this symbol table is empty, and false otherwise
<code>int size()</code>	returns the number of key-value pairs in this symbol table
<code>void put(K key, V value)</code>	inserts the key and value pair into this symbol table
<code>V get(K key)</code>	returns the value associated with key in this symbol table, or null
<code>boolean contains(K key)</code>	returns true if this symbol table contains key , and false otherwise
<code>void delete(K key)</code>	deletes key and the associated value from this symbol table
<code>Iterable<K> keys()</code>	returns all the keys in this symbol table
<code>String toString()</code>	returns a string representation of this symbol table

4.4 BinarySearchST<K extends Comparable<K>, V> implements OrderedST<K, V>

<code>BinarySearchST()</code>	constructs an empty symbol table
<code>boolean isEmpty()</code>	returns true if this symbol table is empty, and false otherwise
<code>int size()</code>	returns the number of key-value pairs in this symbol table
<code>void put(K key, V value)</code>	inserts the key and value pair into this symbol table
<code>V get(K key)</code>	returns the value associated with key in this symbol table, or null
<code>boolean contains(K key)</code>	returns true if this symbol table contains key , and false otherwise
<code>void delete(K key)</code>	deletes key and the associated value from this symbol table

<code>Iterable<K> keys()</code>	returns all the keys in this symbol table in sorted order
<code>K min()</code>	returns the smallest key in this symbol table
<code>K max()</code>	returns the largest key in this symbol table
<code>void deleteMin()</code>	deletes the smallest key and the associated value from this symbol table
<code>void deleteMax()</code>	deletes the largest key and the associated value from this symbol table
<code>K floor(K key)</code>	returns the largest key in this symbol table that is smaller than or equal to key
<code>K ceiling(K key)</code>	returns the smallest key in this symbol table that is greater than or equal to key
<code>int rank(K key)</code>	returns the number of keys in this symbol table that are strictly smaller than key
<code>K select(int k)</code>	returns the key in this symbol table with the rank k
<code>int size(K lo, K hi)</code>	returns the number of keys in this symbol table that are in the interval [lo, hi]
<code>Iterable<K> keys(K lo, K hi)</code>	returns the keys in this symbol table that are in the interval [lo, hi] in sorted order
<code>String toString()</code>	returns a string representation of this symbol table

4.5 SeparateChainingHashST<K, V> implements BasicST<K, V>

<code>SeparateChainingHashST()</code>	constructs an empty symbol table
<code>boolean isEmpty()</code>	returns true if this symbol table is empty, and false otherwise
<code>int size()</code>	returns the number of key-value pairs in this symbol table
<code>void put(K key, V value)</code>	inserts the key and value pair into this symbol table
<code>V get(K key)</code>	returns the value associated with key in this symbol table, or null
<code>boolean contains(K key)</code>	returns true if this symbol table contains key , and false otherwise
<code>void delete(K key)</code>	deletes key and the associated value from this symbol table
<code>Iterable<K> keys()</code>	returns all the keys in this symbol table
<code>String toString()</code>	returns a string representation of this symbol table

4.6 BinarySearchTreeST<K extends Comparable<K>, V> implements OrderedST<K, V>

<code>BinarySearchTreeST()</code>	constructs an empty symbol table
<code>boolean isEmpty()</code>	returns true if this symbol table is empty, and false otherwise
<code>int size()</code>	returns the number of key-value pairs in this symbol table
<code>void put(K key, V value)</code>	inserts the key and value pair into this symbol table
<code>V get(K key)</code>	returns the value associated with key in this symbol table, or null
<code>boolean contains(K key)</code>	returns true if this symbol table contains key , and false otherwise
<code>void delete(K key)</code>	deletes key and the associated value from this symbol table
<code>Iterable<K> keys()</code>	returns all the keys in this symbol table in sorted order
<code>K min()</code>	returns the smallest key in this symbol table
<code>K max()</code>	returns the largest key in this symbol table
<code>void deleteMin()</code>	deletes the smallest key and the associated value from this symbol table
<code>void deleteMax()</code>	deletes the largest key and the associated value from this symbol table
<code>K floor(K key)</code>	returns the largest key in this symbol table that is smaller than or equal to key
<code>K ceiling(K key)</code>	returns the smallest key in this symbol table that is greater than or equal to key

<code>int rank(K key)</code>	returns the number of keys in this symbol table that are strictly smaller than key
<code>K select(int k)</code>	returns the key in this symbol table with the rank k
<code>int size(K lo, K hi)</code>	returns the number of keys in this symbol table that are in the interval <code>[lo, hi]</code>
<code>Iterable<K> keys(K lo, K hi)</code>	returns the keys in this symbol table that are in the interval <code>[lo, hi]</code> in sorted order
<code>String toString()</code>	returns a string representation of this symbol table
<code>Iterable<K> preOrder()</code>	returns all the keys from this symbol table in “pre” order
<code>Iterable<K> inOrder()</code>	returns all the keys from this symbol table in “in” order
<code>Iterable<K> postOrder()</code>	returns all the keys from this symbol table in “post” order

4.7 RedBlackBinarySearchTreeST<K extends Comparable<K>, V> implements OrderedST<K, V>

<code>RedBlackBinarySearchTreeST()</code>	constructs an empty symbol table
<code>boolean isEmpty()</code>	returns true if this symbol table is empty, and false otherwise
<code>int size()</code>	returns the number of key-value pairs in this symbol table
<code>void put(K key, V value)</code>	inserts the key and value pair into this symbol table
<code>V get(K key)</code>	returns the value associated with key in this symbol table, or null
<code>boolean contains(K key)</code>	returns true if this symbol table contains key , and false otherwise
<code>void delete(K key)</code>	deletes key and the associated value from this symbol table
<code>Iterable<K> keys()</code>	returns all the keys in this symbol table in sorted order
<code>K min()</code>	returns the smallest key in this symbol table
<code>K max()</code>	returns the largest key in this symbol table
<code>void deleteMin()</code>	deletes the smallest key and the associated value from this symbol table
<code>void deleteMax()</code>	deletes the largest key and the associated value from this symbol table
<code>K floor(K key)</code>	returns the largest key in this symbol table that is smaller than or equal to key
<code>K ceiling(K key)</code>	returns the smallest key in this symbol table that is greater than or equal to key
<code>int rank(K key)</code>	returns the number of keys in this symbol table that are strictly smaller than key
<code>K select(int k)</code>	returns the key in this symbol table with the rank k
<code>int size(K lo, K hi)</code>	returns the number of keys in this symbol table that are in the interval <code>[lo, hi]</code>
<code>Iterable<K> keys(K lo, K hi)</code>	returns the keys in this symbol table that are in the interval <code>[lo, hi]</code> in sorted order
<code>String toString()</code>	returns a string representation of this symbol table
<code>Iterable<K> preOrder()</code>	returns all the keys from this symbol table in “pre” order
<code>Iterable<K> inOrder()</code>	returns all the keys from this symbol table in “in” order
<code>Iterable<K> postOrder()</code>	returns all the keys from this symbol table in “post” order

4.8 Set<K extends Comparable<K>> implements Iterable<K>

<code>Set()</code>	constructs an empty set
<code>boolean isEmpty()</code>	returns true if this set is empty, and false otherwise
<code>int size()</code>	returns the number of keys in this set
<code>void add(K key)</code>	adds key to this set, if it is not already present

<code>boolean contains(K key)</code>	returns <code>true</code> if this set contains <code>key</code> , and <code>false</code> otherwise
<code>void delete(K key)</code>	deletes <code>key</code> from this set
<code>Iterator<K> iterator()</code>	returns an iterator to iterate over the keys in this set in sorted order
<code>String toString()</code>	returns a string representation of this set

4.9 SparseVector

<code>SparseVector(int n)</code>	constructs an <code>n</code> -dimensional zero vector
<code>int dimension()</code>	returns the dimension of this vector
<code>int size()</code>	returns the number of nonzero entries in this vector
<code>void put(int i, double value)</code>	sets the <code>i</code> th component of this vector to <code>value</code>
<code>double get(int i)</code>	returns the <code>i</code> th component of this vector
<code>SparseVector plus(SparseVector other)</code>	returns the sum of this vector and <code>other</code>
<code>SparseVector scale(double alpha)</code>	returns the scalar-vector product of this vector and <code>alpha</code>
<code>double dot(SparseVector other)</code>	returns the dot product of this vector and <code>other</code>
<code>double magnitude()</code>	returns the magnitude of this vector
<code>String toString()</code>	returns a string representation of this vector

4.10 SparseMatrix

<code>SparseMatrix(int m, int n)</code>	constructs an <code>m x n</code> dimensional zero matrix
<code>int nRows()</code>	returns the number of rows in this matrix
<code>int nCols()</code>	returns the number of columns in this matrix
<code>int size()</code>	returns the number of nonzero entries in this matrix
<code>void put(int i, int j, double value)</code>	sets the entry at row <code>i</code> and column <code>j</code> in this matrix to <code>value</code>
<code>double get(int i, int j)</code>	returns the entry in this matrix at row <code>i</code> and column <code>j</code>
<code>SparseMatrix plus(SparseMatrix other)</code>	returns the sum of this matrix and <code>other</code>
<code>SparseVector times(SparseVector x)</code>	returns the product of this matrix and the vector <code>x</code>
<code>String toString()</code>	returns a string representation of this matrix

5 Graphs

5.1 Graph

<code>Graph(int V)</code>	constructs an empty graph with <code>V</code> vertices and 0 edges
<code>Graph(In in)</code>	constructs a graph from the input stream <code>in</code>
<code>int V()</code>	returns the number of vertices in this graph
<code>int E()</code>	returns the number of edges in this graph
<code>void addEdge(int v, int w)</code>	adds an undirected edge between vertices <code>v</code> and <code>w</code> in this graph
<code>Iterable<Integer> adj(int v)</code>	returns the vertices adjacent to vertex <code>v</code> in this graph
<code>int degree(int v)</code>	returns the degree of vertex <code>v</code> in this graph
<code>String toString()</code>	returns a string representation of this graph

5.2 interface Paths

<code>boolean hasPathTo(int v)</code>	returns true if there is a path between a designated source vertex and vertex <code>v</code> , and false otherwise
<code>Iterable<Integer> pathTo(int v)</code>	returns a path between a designated source vertex and vertex <code>v</code> , or <code>null</code>
<code>double distTo(int v)</code>	returns the shortest distance between a designated source vertex and vertex <code>v</code> , or ∞

5.3 DFSPaths implements Paths

<code>DFSPaths(Graph G, int s)</code>	computes paths between source vertex <code>s</code> and every other vertex in the graph <code>G</code>
<code>boolean hasPathTo(int v)</code>	returns true if there is a path between a designated source vertex and vertex <code>v</code> , and false otherwise
<code>Iterable<Integer> pathTo(int v)</code>	returns a path between a designated source vertex and vertex <code>v</code> , or <code>null</code>

5.4 BFSPaths implements Paths

<code>BFSPaths(Graph G, int s)</code>	computes shortest paths between source vertex <code>s</code> and every other vertex in the graph <code>G</code>
<code>boolean hasPathTo(int v)</code>	returns true if there is a path between a designated source vertex and vertex <code>v</code> , and false otherwise
<code>Iterable<Integer> pathTo(int v)</code>	returns a path between a designated source vertex and vertex <code>v</code> , or <code>null</code>
<code>double distTo(int v)</code>	returns the shortest distance between a designated source vertex and vertex <code>v</code> , or ∞

5.5 SymbolGraph

<code>SymbolGraph(In in, String delim)</code>	constructs a symbol graph from the input stream <code>in</code> and using <code>delim</code> as the delimiter
<code>boolean contains(String s)</code>	returns true if this symbol graph contains vertex <code>s</code> , and false otherwise
<code>int indexOf(String s)</code>	returns the integer associated with the vertex <code>s</code> in this symbol graph
<code>String nameOf(int v)</code>	returns the name of the vertex associated with the integer <code>v</code> in this symbol graph
<code>Graph graph()</code>	returns the graph associated with this symbol graph
<code>String toString()</code>	returns a string representation of this symbol graph

5.6 DiGraph

<code>DiGraph(int V)</code>	constructs an empty digraph with <code>V</code> vertices and 0 edges
<code>DiGraph(In in)</code>	constructs a digraph from the input stream <code>in</code>
<code>int V()</code>	returns the number of vertices in this digraph
<code>int E()</code>	returns the number of edges in this digraph
<code>void addEdge(int v, int w)</code>	adds the directed edge <code>v→w</code> to this digraph
<code>Iterable<Integer> adj(int v)</code>	returns the vertices adjacent from vertex <code>v</code> in this digraph
<code>int outDegree(int v)</code>	returns the out-degree of vertex <code>v</code> in this digraph

<code>int inDegree(int v)</code>	returns the in-degree of vertex <code>v</code> in this digraph
<code>String toString()</code>	returns a string representation of this digraph

5.7 DFSDiPaths implements Paths

<code>DFSDiPaths(DiGraph G, int s)</code>	computes paths from source vertex <code>s</code> to every other vertex in the digraph <code>G</code>
<code>boolean hasPathTo(int v)</code>	returns <code>true</code> if there is a path between a designated source vertex and vertex <code>v</code> , and <code>false</code> otherwise
<code>Iterable<Integer> pathTo(int v)</code>	returns a path between a designated source vertex and vertex <code>v</code> , or <code>null</code>

5.8 BFSDiPaths implements Paths

<code>BFSDiPaths(DiGraph G, int s)</code>	computes shortest paths from source vertex <code>s</code> to every other vertex in the digraph <code>G</code>
<code>boolean hasPathTo(int v)</code>	returns <code>true</code> if there is a path between a designated source vertex and vertex <code>v</code> , and <code>false</code> otherwise
<code>Iterable<Integer> pathTo(int v)</code>	returns a path between a designated source vertex and vertex <code>v</code> , or <code>null</code>
<code>double distTo(int v)</code>	returns the shortest distance between a designated source vertex and vertex <code>v</code> , or ∞

5.9 SymbolDiGraph

<code>SymbolDiGraph(In in, String delim)</code>	constructs a symbol digraph from the input stream <code>in</code> and using <code>delim</code> as the delimiter
<code>boolean contains(String s)</code>	returns <code>true</code> if this symbol digraph contains vertex <code>s</code> , and <code>false</code> otherwise
<code>int indexOf(String s)</code>	returns the integer associated with the vertex <code>s</code> in this symbol digraph
<code>String nameOf(int v)</code>	returns the name of the vertex associated with the integer <code>v</code> in this symbol digraph
<code>DiGraph diGraph()</code>	returns the digraph associated with this symbol digraph
<code>String toString()</code>	returns a string representation of this symbol digraph

5.10 DiCycle

<code>DiCycle(DiGraph G)</code>	determines whether the digraph <code>G</code> has a directed cycle and, if so, finds such a cycle
<code>boolean hasCycle()</code>	returns <code>true</code> if a directed cycle was detected, and <code>false</code> otherwise
<code>Iterable<Integer> cycle()</code>	returns a directed cycle, or <code>null</code>

5.11 DFSOrders

<code>DFSOrders(DiGraph G)</code>	determines depth-first orders (pre, post, and reverse post) for the digraph <code>G</code>
<code>int pre(int v)</code>	returns the pre-order number of vertex <code>v</code>
<code>int post(int v)</code>	returns the post-order number of vertex <code>v</code>
<code>Iterable<Integer> pre()</code>	returns the vertices in pre-order

<code>Iterable<Integer> post()</code>	returns the vertices in post-order
<code>Iterable<Integer> reversePost()</code>	returns the vertices in reverse post-order

5.12 Topological

<code>Topological(DiGraph G)</code>	determines whether the digraph G has a topological order and, if so, finds such an order
<code>boolean hasOrder()</code>	returns true if there exists a topological order, and false otherwise
<code>Iterable<Integer> order()</code>	returns a topological order, or null

5.13 Edge implements Comparable<Edge>

<code>Edge(int v, int w, double weight)</code>	constructs an edge between vertices v and w of the given weight
<code>int either()</code>	returns one endpoint of this edge
<code>int other(int v)</code>	returns the endpoint of this edge that is different from vertex v
<code>double weight()</code>	returns the weight of this edge
<code>String toString()</code>	returns a string representation of this edge
<code>int compareTo(Edge other)</code>	returns a comparison of this edge with other by their weights

5.14 EdgeWeightedGraph

<code>EdgeWeightedGraph(int V)</code>	constructs an empty edge-weighted graph with V vertices and 0 edges
<code>EdgeWeightedGraph(In in)</code>	constructs an edge-weighted graph from the input stream in
<code>int V()</code>	returns the number of vertices in this edge-weighted graph
<code>int E()</code>	returns the number of edges in this edge-weighted graph
<code>void addEdge(Edge e)</code>	adds an edge e to this edge-weighted graph
<code>Iterable<Integer> adj(int v)</code>	returns the edges incident on vertex v in this edge-weighted graph
<code>int degree(int v)</code>	returns the degree of vertex v in this edge-weighted graph
<code>Iterable<Edge> edges()</code>	returns all the edges in this edge-weighted graph
<code>String toString()</code>	returns a string representation of this edge-weighted graph

5.15 Kruskal

<code>Kruskal(EdgeWeightedGraph G)</code>	determines the minimum spanning tree (MST) of the edge-weighted graph G
<code>Iterable<Edge> edges()</code>	returns the edges in the MST
<code>double weight()</code>	returns the sum of the edge weights in the MST

5.16 DiEdge

<code>DiEdge(int v, int w, double weight)</code>	constructs a directed edge from vertex v to vertex w of the given weight
<code>int from()</code>	returns the tail vertex of this directed edge
<code>int to()</code>	returns the head vertex of this directed edge
<code>double weight()</code>	returns the weight of this directed edge
<code>String toString()</code>	returns a string representation of this directed edge

5.17 EdgeWeightedDiGraph

<code>EdgeWeightedDiGraph(int V)</code>	constructs an empty edge-weighted digraph with <code>V</code> vertices and 0 edges
<code>EdgeWeightedDiGraph(In in)</code>	constructs an edge-weighted digraph from the input stream <code>in</code>
<code>int V()</code>	returns the number of vertices in this edge-weighted digraph
<code>int E()</code>	returns the number of edges in this edge-weighted digraph
<code>void addEdge(DiEdge e)</code>	adds a directed edge <code>e</code> to this edge-weighted digraph
<code>Iterable<Integer> adj(int v)</code>	returns the directed edges incident from vertex <code>v</code> in this edge-weighted digraph
<code>int outDegree(int v)</code>	returns the out-degree of vertex <code>v</code> in this edge-weighted digraph
<code>int inDegree(int v)</code>	returns the in-degree of vertex <code>v</code> in this edge-weighted digraph
<code>Iterable<DiEdge> edges()</code>	returns all the directed edges in this edge-weighted digraph
<code>String toString()</code>	returns a string representation of this edge-weighted digraph

5.18 Dijkstra implements Paths

<code>Dijkstra(DiGraph G, int s)</code>	determines the shortest paths from the source vertex <code>s</code> to every other vertex in the edge-weighted digraph <code>G</code>
<code>boolean hasPathTo(int v)</code>	returns <code>true</code> if there is a path between a designated source vertex and vertex <code>v</code> , and <code>false</code> otherwise
<code>Iterable<Integer> pathTo(int v)</code>	returns a path between a designated source vertex and vertex <code>v</code> , or <code>null</code>
<code>double distTo(int v)</code>	returns the shortest distance between a designated source vertex and vertex <code>v</code> , or ∞

6 Strings

6.1 Alphabet

<code>static Alphabet BINARY</code>	the binary alphabet $\{0, 1\}$
<code>static Alphabet OCTAL</code>	the octal alphabet $\{0, 1, 2, 3, 4, 5, 6, 7\}$
<code>static Alphabet DECIMAL</code>	the decimal alphabet $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
<code>static Alphabet HEXADECIMAL</code>	the hexadecimal alphabet $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F\}$
<code>static Alphabet DNA</code>	the DNA alphabet $\{A, C, G, T\}$
<code>static Alphabet LOWERCASE</code>	the lowercase alphabet $\{a, b, c, \dots, z\}$
<code>static Alphabet UPPERCASE</code>	the uppercase alphabet $\{A, B, C, \dots, Z\}$
<code>static Alphabet PROTEIN</code>	the protein alphabet $\{A, C, D, E, F, G, H, I, K, L, M, N, P, Q, R, S, T, V, W, Y\}$
<code>static Alphabet BASE64</code>	the base-64 alphabet (64 characters)
<code>static Alphabet ASCII</code>	the ASCII alphabet (0 – 127)
<code>static Alphabet EXTENDED_ASCII</code>	the extended ASCII alphabet (0 – 255)
<code>static Alphabet UNICODE16</code>	the Unicode 16 alphabet (0 – 65,535)
<code>Alphabet()</code>	constructs a new alphabet using characters 0 through 255
<code>Alphabet(int radix)</code>	constructs a new alphabet using characters 0 through <code>radix - 1</code>
<code>Alphabet(String s)</code>	constructs a new alphabet from the string of characters <code>s</code>
<code>boolean contains(char c)</code>	returns <code>true</code> if <code>c</code> is a character in this alphabet, and <code>false</code> otherwise

<code>int radix()</code>	returns the radix of this alphabet
<code>int lgRadix()</code>	returns the binary logarithm (rounded up) of this alphabet's radix
<code>int toIndex(char c)</code>	returns the index of <code>c</code>
<code>int[] toIndices(String s)</code>	returns the indices of the characters in <code>s</code>
<code>char toChar(int index)</code>	returns the character with the given index
<code>String toChars(int[] indices)</code>	returns the characters with the given indices

6.2 LSD

<code>static void sort(String[] a)</code>	sorts the array <code>a</code> of fixed-length strings over the extended ASCII alphabet
---	---

6.3 MSD

<code>static void sort(String[] a)</code>	sorts the array <code>a</code> of strings over the extended ASCII alphabet
---	--

6.4 TrieST

<code>TrieST()</code>	constructs an empty symbol table
<code>boolean isEmpty()</code>	returns true if this symbol table is empty, and false otherwise
<code>int size()</code>	returns the number of key-value pairs in this symbol table
<code>void put(String key, V value)</code>	inserts the key and value pair into this symbol table
<code>V get(String key)</code>	returns the value associated with key in this symbol table, or null
<code>boolean contains(String key)</code>	returns true if this symbol table contains key , and false otherwise
<code>void delete(String key)</code>	deletes key and the associated value from this symbol table
<code>Iterable<String> keys()</code>	returns all the keys in this symbol table
<code>Iterable<String> keysWithPrefix(String prefix)</code>	returns all the keys in this symbol table that start with prefix
<code>Iterable<String> keysThatMatch(String pattern)</code>	returns all the keys in this symbol table that match pattern , where the <code>.</code> symbol is treated as a wildcard character
<code>Iterable<String> longestPrefixOf(String query)</code>	returns the string in this symbol table that is the longest prefix of query , or null
<code>String toString()</code>	returns a string representation of this symbol table

6.5 KMP

<code>KMP(String pattern, int radix)</code>	preprocesses the pattern string with alphabet size given by radix
<code>int search(String text)</code>	returns the index of the first occurrence of the pattern string within the text string, or the length of the text string

6.6 NFA

<code>NFA(String regexp)</code>	constructs a nondeterministic finite state automaton (NFA) from regexp
<code>boolean recognizes(String text)</code>	returns true if this NFA recognizes text , and false otherwise

6.7 Genome

<code>static void compress()</code>	reads from standard input a sequence characters over the alphabet $\{A, C, G, T\}$; compresses them using two bits per character; and writes the results to standard output
<code>static void expand()</code>	reads from standard input a sequence of genome-compressed bits; expands each two bits into a character over the alphabet $\{A, C, G, T\}$; and writes the results to standard output

6.8 RunLength

<code>static void compress()</code>	reads from standard input a sequence of bits; compresses them using run-length coding with 8-bit run lengths; and writes the results to standard output
<code>static void expand()</code>	reads from standard input a sequence of runlength-compressed bits; expands them; and writes the results to standard output

6.9 Huffman

<code>static void compress()</code>	reads from standard input a sequence of bytes; compresses them using Huffman codes with an 8-bit alphabet; and writes the results to standard output
<code>static void expand()</code>	reads from standard input a sequence of Huffman-compressed bits; expands them; and writes the results to standard output